
VacayOps

Conception, Use Cases, Data Model & Database Design

USE CASE DIAGRAM · CLASS DIAGRAM · UX FLOW · SCHEMA

Table of Contents

1.	How VacayOps Works -- System Overview
2.	Use Case Diagram
3.	Class Diagram & Data Model
4.	Database Schema -- Bare Minimum, Built to Expand
5.	UX Flow & Initial Page Structure
6.	Technology Recommendations
7.	Next Steps for the Team

1. HOW VACAYOPS WORKS -- SYSTEM OVERVIEW

VacayOps has six actors: three are people who log in and use the platform directly, one is a person who never logs in at all, one is the system itself acting autonomously, and two are external platforms the system talks to automatically.

Actor	Type	Role
Property Manager	Human, logs in	Oversees all properties, views the dashboard, assigns tickets, manages the team
Staff Member	Human, logs in	Handles day-to-day guest conversations and maintenance coordination
Contractor	Human, logs in (limited)	Receives ticket assignments, updates their status -- no access beyond their own tickets
Guest	Human, never logs in	Books and stays at a property; communicates only through WhatsApp
AI Assistant	System actor	Reads guest messages, replies automatically or escalates to a human
Booking Platforms	External system	Airbnb / Booking.com -- source of booking data via API or iCal feed

The three flows that make up the product

Booking Sync Flow

A guest books on Airbnb. Airbnb sends the booking data to VacayOps through its API or iCal feed. The system creates a Booking record linked to that property's Channel. It immediately checks the new dates against every other channel for the same property. If two channels claim overlapping dates, the Property Manager is alerted before it becomes a real double-booking. The portfolio calendar updates for the whole team in real time.

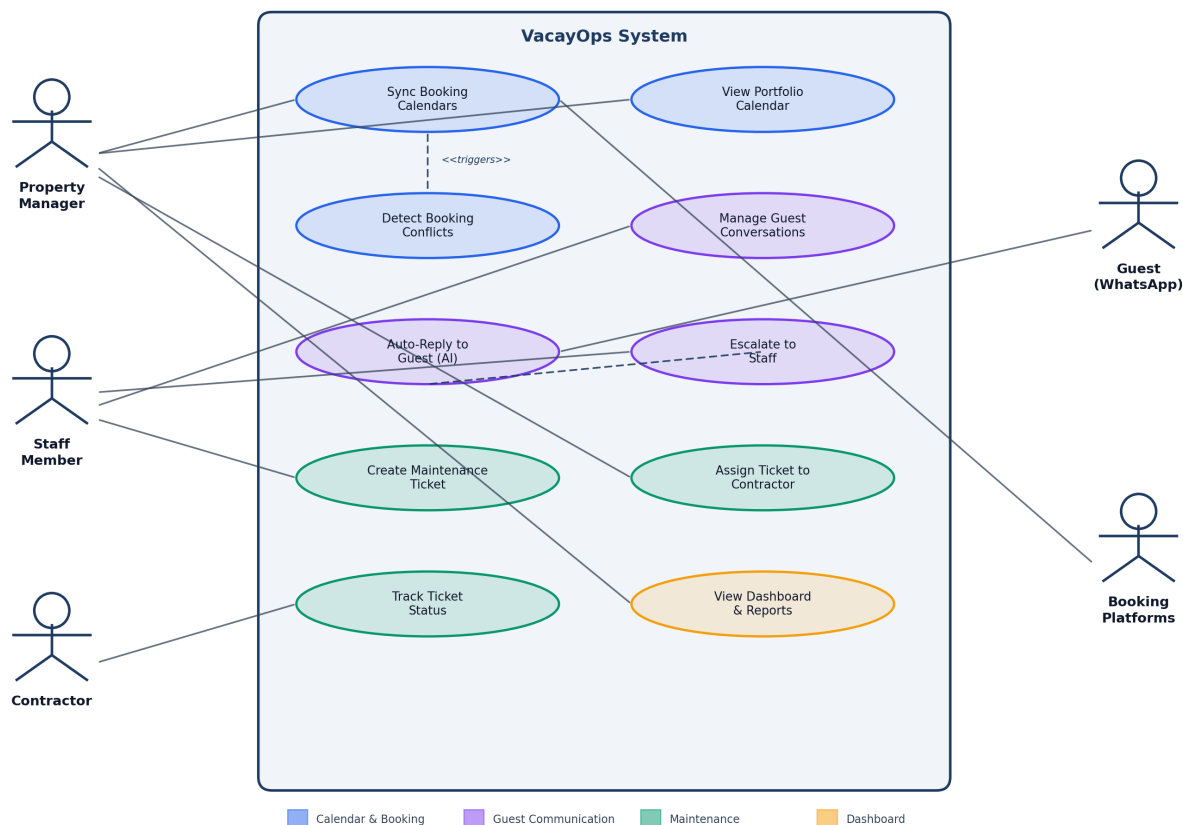
Guest Messaging Flow

A guest sends a WhatsApp message. It arrives through the WhatsApp Business API webhook. The system finds or creates the Conversation for that guest and property. The AI Assistant reads the message together with that property's knowledge card (WiFi, check-in steps, house rules). Routine questions get an automatic reply. Anything outside its scope -- a complaint, a refund request -- is flagged, and a Staff Member is notified to take over with full context already loaded. Every message, AI-written or human, is saved to keep one full history per guest.

Maintenance Ticket Flow

A guest or staff member reports an issue -- a broken air conditioner, for example. A Ticket is created, linked to the property and, if relevant, the guest's active Booking. It is assigned to the right Contractor based on the type of issue. The contractor is notified and updates the status as they work. The guest gets an automatic update. Once resolved, the ticket closes and stays in that property's maintenance history for future reference.

2. USE CASE DIAGRAM



Solid lines: an actor directly triggers a use case. Dashed lines: one use case automatically triggers another inside the system.

What each use case actually means

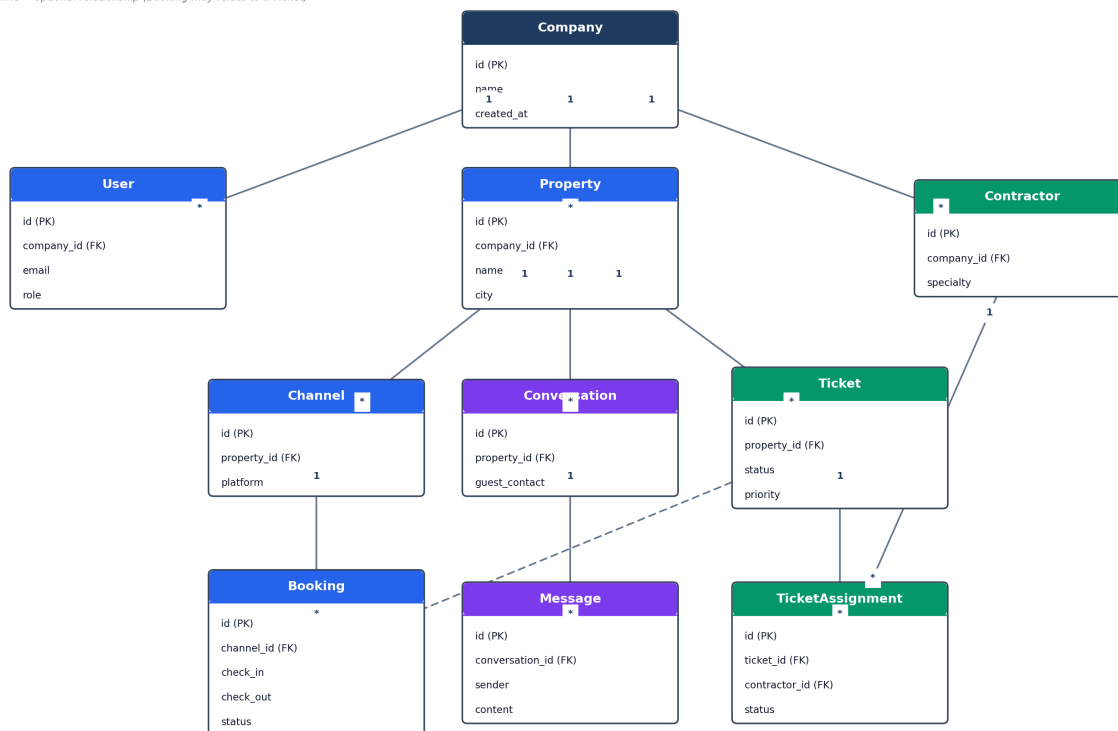
Use Case	Primary Actor	Trigger	Outcome
Sync Booking Calendars	Booking Platforms	A booking is made or changed on Airbnb/Booking.com	New Booking record created or updated
Detect Booking Conflicts	(system, triggered)	A new Booking overlaps existing dates on another channel	Property Manager alerted immediately
View Portfolio Calendar	Property Manager	User opens the Calendar page	All properties, all channels, one screen
Manage Guest Conversations	Staff Member	A conversation needs human attention	Staff replies, full history visible
Auto-Reply to Guest (AI)	Guest (via WhatsApp)	Guest sends a routine question	AI answers using the property's knowledge card
Escalate to Staff	(system, triggered)	AI cannot confidently answer	Conversation flagged, staff notified with context
Create Maintenance Ticket	Staff Member	An issue is reported	New Ticket record created, status = open

Assign Ticket to Contractor	Property Manager	A ticket needs a contractor	Contractor notified, ticket status = assigned
Track Ticket Status	Contractor	Work is in progress or done	Status updated, guest notified automatically
View Dashboard & Reports	Property Manager	User opens the Dashboard	Occupancy, open tickets, unread messages at a glance

3. CLASS DIAGRAM & DATA MODEL

This is the bare-minimum data model -- the smallest set of entities that supports every use case above without leaving out anything essential. Each entity below shows only its most important fields for readability; the complete field list is in Section 4's SQL schema.

Dashed line = optional relationship (Booking may relate to a Ticket)



Every entity belongs to a Company -- this makes the system multi-tenant from day one, so a second property management company could use the same platform later without a schema change.

Entity	What it represents
Company	The property management company itself. Everything else belongs to one.
User	A person who logs in -- Property Manager, Staff, or Contractor, distinguished by role.
Property	One villa or apartment being managed.
Channel	One property's presence on one booking platform (its Airbnb listing, its Booking.com listing).
Booking	One guest's reservation, arriving through a specific Channel.
Conversation	One ongoing WhatsApp thread with one guest, tied to a property.
Message	One message inside a Conversation -- from the guest, the AI, or a staff member.
Ticket	One maintenance issue, tied to a property and optionally to the guest's active Booking.
Contractor	One external service provider -- plumber, electrician, cleaner, handyman.
TicketAssignment	The link between a Ticket and the Contractor handling it, with its own status.

4. DATABASE SCHEMA -- BARE MINIMUM, BUILT TO EXPAND

Recommended database: PostgreSQL

PostgreSQL over MySQL or a NoSQL option, for three concrete reasons specific to this project: the data is inherently relational -- a Booking must always trace back to a real Property and Channel, and that referential integrity matters when the whole point of the product is preventing double-bookings. Second, PostgreSQL's native JSONB type gives room to store flexible data (a property's knowledge card, AI conversation metadata) without a rigid schema fight later. Third, it's the database already specified in your tech stack and Docker setup -- no new decision needed here, just confirming it's the right one.

The schema -- ready to run

This creates exactly the entities in the class diagram, nothing more. It intentionally leaves out fields you'll want later (pricing rules, review scores, multi-language content) -- add those as columns when a real feature needs them, not before.

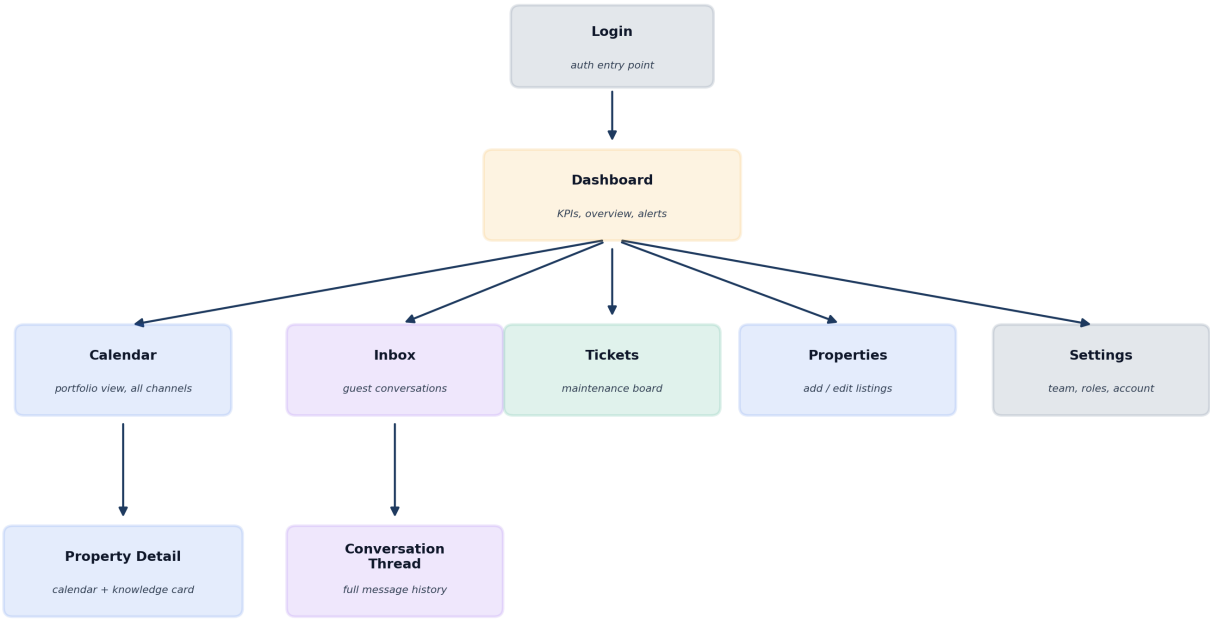
```
CREATE TABLE company (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE app_user (  
    id SERIAL PRIMARY KEY,  
    company_id INTEGER NOT NULL REFERENCES company(id),  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    role VARCHAR(50) NOT NULL DEFAULT 'staff',  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE property (  
    id SERIAL PRIMARY KEY,  
    company_id INTEGER NOT NULL REFERENCES company(id),  
    name VARCHAR(255) NOT NULL,  
    address VARCHAR(255),  
    city VARCHAR(100),  
    wifi_password VARCHAR(255),  
    checkin_instructions TEXT,  
    house_rules TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE channel (  
    id SERIAL PRIMARY KEY,  
    property_id INTEGER NOT NULL REFERENCES property(id),  
    platform VARCHAR(50) NOT NULL, -- airbnb, booking, expedia  
    external_listing_id VARCHAR(255),  
    ical_url TEXT  
);  
  
CREATE TABLE booking (  
    id SERIAL PRIMARY KEY,  
    channel_id INTEGER NOT NULL REFERENCES channel(id),  
    guest_name VARCHAR(255),  
    guest_contact VARCHAR(255),  
    check_in DATE NOT NULL,  
    check_out DATE NOT NULL,  
    status VARCHAR(50) DEFAULT 'confirmed',  
    external_booking_id VARCHAR(255),  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE conversation (  
    id SERIAL PRIMARY KEY,  
    property_id INTEGER NOT NULL REFERENCES property(id),  
    guest_contact VARCHAR(255) NOT NULL,  
    channel VARCHAR(50) DEFAULT 'whatsapp',  
    status VARCHAR(50) DEFAULT 'open',  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE message (  
    id SERIAL PRIMARY KEY,  
    conversation_id INTEGER NOT NULL REFERENCES conversation(id),  
    sender VARCHAR(20) NOT NULL, -- guest, ai, staff  
    content TEXT NOT NULL,  
    language VARCHAR(10),  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE ticket (  
    id SERIAL PRIMARY KEY,  
    property_id INTEGER NOT NULL REFERENCES property(id),  
    booking_id INTEGER REFERENCES booking(id), -- nullable, optional  
    title VARCHAR(255) NOT NULL,  
    description TEXT,  
    priority VARCHAR(20) DEFAULT 'normal', -- urgent, normal, low  
    status VARCHAR(30) DEFAULT 'open', -- open, assigned, in_progress, resolved  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE contractor (  
    id SERIAL PRIMARY KEY,  
    company_id INTEGER NOT NULL REFERENCES company(id),  
    name VARCHAR(255) NOT NULL,  
    phone VARCHAR(50),  
    specialty VARCHAR(100) -- plumber, electrician, cleaner, handyman  
);  
  
CREATE TABLE ticket_assignment (  
    id SERIAL PRIMARY KEY,  
    ticket_id INTEGER NOT NULL REFERENCES ticket(id),  
    contractor_id INTEGER NOT NULL REFERENCES contractor(id),  
    assigned_at TIMESTAMP DEFAULT NOW(),  
    status VARCHAR(30) DEFAULT 'assigned'  
);
```

*Note: **user** is a reserved word in PostgreSQL, so the table is named **app_user** instead -- using plain "user" will throw a syntax error the first time someone runs this.*

5. UX FLOW & INITIAL PAGE STRUCTURE

Navbar navigation: Dashboard, Calendar, Inbox, Tickets, Properties, and Settings are always reachable from anywhere -- not a strict one-way flow.



Page	Purpose	Key elements	Primary entities
Login	Authenticate	Email/password form	app_user
Dashboard	Portfolio-level overview	KPI cards, alerts, quick links	Property, Booking, Ticket, Conversation (aggregated)
Calendar	See every booking, every channel	Multi-property calendar grid, conflict alerts	Property, Channel, Booking
Property Detail	Manage one property	Its calendar, knowledge-card editor, channel list	Property, Channel, Booking
Properties	Add / manage properties	List view + create/edit form	Property
Inbox	Handle guest conversations	Conversation list, AI/human indicator	Conversation, Message
Conversation Thread	Full history with one guest	Chat-style view, take-over button	Message
Tickets	Maintenance board	Kanban: Open / Assigned / In Progress / Resolved	Ticket, TicketAssignment, Contractor
Settings	Manage team & account	User list, roles, company info	app_user, Company

6. TECHNOLOGY RECOMMENDATIONS

Database: PostgreSQL 15+ (confirmed above)

Relational, referential integrity by default, JSONB for the flexible fields you'll add later. Already in your Docker setup.

UI: React + TypeScript + TailwindCSS + shadcn/ui

React and TypeScript were already decided in your project report and match your team's frontend role. Two additions worth locking in now, specifically because of the page structure above: TailwindCSS for styling, since a Kanban board (Tickets), a calendar grid (Calendar), and a chat thread (Conversation Thread) all need custom layouts that pre-built component kits fight against. shadcn/ui on top of that for the pieces that don't need custom design -- forms, buttons, dropdowns, the login page -- so nobody spends time rebuilding a text input from scratch.

Recommended libraries for the two hardest UI pieces

Piece	Recommended library	Why
Calendar grid	FullCalendar (React) or a custom grid on top of date-fns	A generic date-picker library won't show multi-property, multi-channel bookings well -- FullCalendar is built for exactly this.
Kanban ticket board	dnd-kit	Modern, actively maintained drag-and-drop -- lighter and more reliable than older alternatives for this use case.

7. NEXT STEPS FOR THE TEAM

This document is a starting draft, not a finished decision. Before any feature branch gets opened against this schema, all three of you need to sit down together -- roughly 90 minutes -- and either confirm or change what's here. That session is where this becomes the team's schema, not one person's proposal.

- Walk through the three flows in Section 1 together -- does everyone agree this is actually how the product should behave?
- Walk through every table in Section 4's SQL -- rename anything that doesn't match how you'll talk about it day to day.
- Confirm the page list in Section 5 -- add or cut a page now, while it costs one sentence, not a rebuilt route later.
- Once agreed, one person runs the SQL against the Postgres container and confirms all ten tables exist with `\dt` in psql.`
- Commit the finalized schema to ``docs/architecture.md`` in the repo, so it's the source of truth everyone builds against.